

ASSIGNMENT [FEB 7 2025]

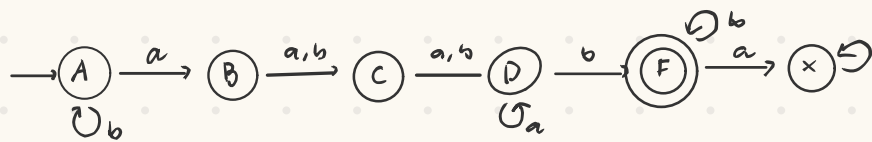
NAME PRINCESS BEVERLY R ALA-AN

COURSE . CSC 109

SECTION . K1

PART I. Design a DFA that recognizes the language Provide a complete DFA definition per item with state machine

1 Strings over $\{a, b\}$ that start with 'a' and end with 'b'



$Q = \{A, B, C, D, F, X\}$

$\Sigma = \{a, b\}$

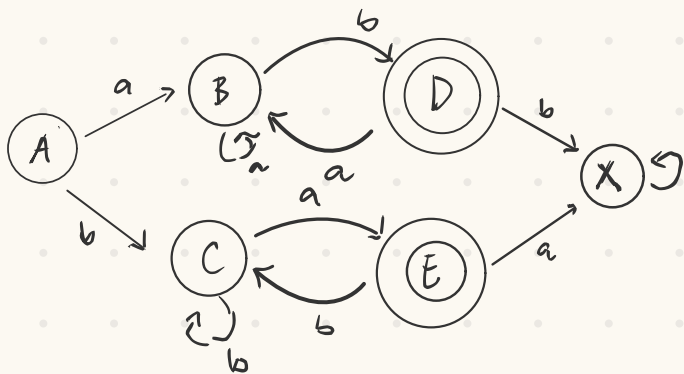
$q_0 = A$

$F = \{F\}$

$\delta: Q \times \Sigma =$ (it is defined by this table)

	A	B	C	D	F	X
a	B	C	D	D	X	Φ
b	A	C	D	F	F	

2. Strings over $\{a, b\}$ that alternate between 'a' and 'b' (eg., "ababab", "babab")



$Q = \{A, B, C, D, E, X\}$

$\Sigma = \{a, b\}$

$q_0 = A$

$F = \{D, E\}$

$\delta: Q \times \Sigma =$ (it is defined by the table)

	A	B	C	D	E	X
a	B	B	E	B	X	Φ
b	C	D	C	X	C	Φ

Checking strings:

ababab ✓

bababa ✓

PART II DEBUGGING ACTIVITY

To-dos

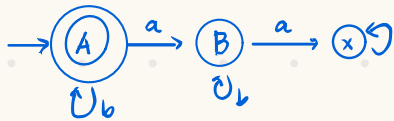
- With the intended language and the described DFA model, spot and correct the errors found in the model. Provided the complete DFA definition of the new model
- With a sample accepted string, use the extended transition to show that the corrected model can accept the string.
- Informally describe the language of the described state machine

1 Language Strings over $\{a, b\}$ that do not contain the substring "aa"

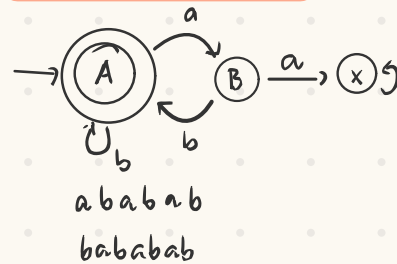
DESCRIPTION of the STATE MACHINE

State 1 is the start state and the accepting state. It transitions to state 2 on input 'a' and to itself on input 'b'. State 2 transitions to itself on input 'b' and is a trap state on input 'a'.

State machine as described



Debugged State Machine



$$Q = \{A, B, X\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = A$$

$$F = \{A\}$$

$$\delta: Q \times \Sigma \rightarrow Q$$

	a	b	x
a	B	X	$\emptyset$
b	A	A	

My description on the debugged state machine

① First state (A) is both the initial and accepting state;

If the input is either 'a' or 'b', it will be accepted by state

Since, we want to avoid having substring "aa", state A only transitions input if it's 'a' otherwise,

input 'b' stays in the accepting state

if input a transitions to state B $\rightarrow a$

else stays in state A $\rightarrow b$

Possible strings bba, ba, bbba

② Second state (B) transitions input 'b' back to accepting stage

Otherwise, input 'a' is sent to state x which is a dead state and the string becomes invalid. This way of handling is strict to prevent substring "aa"

if input 'b' transitions to state A $\rightarrow a-b$

else transitions to dead state $\rightarrow aa \times$

possible initial string bbab, ab, bab

③ What will happen later on is that the transitions will loop between state A and B. And the string will always

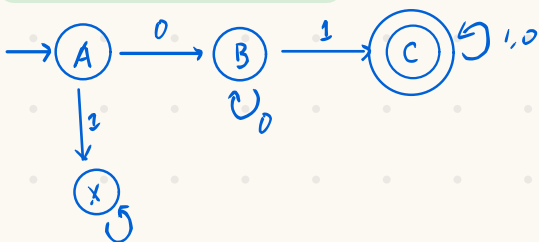
end with b otherwise, the string is invalid. Possible strings accepted by the state machine: ✓ abababab ✓ bbababab ✓ babab

2 Language Strings over $\{0, 1\}$ that start with '0' and have at least two '1's

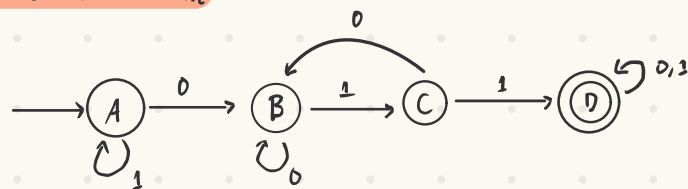
DESCRIPTION of the STATE MACHINE

State 1 is the start state. It transitions to State 2 on input '0' and is a trap state on input '1'. State 2 transitions to State 3 on input '1', and to itself on input '0'. State 3 is the accepting and transitions to itself on both inputs.

State machine as described



Debugged State Machine



$$Q = \{A, B, C, D\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = A$$

$$F = \{C\}$$

$$\delta: Q \times \Sigma \rightarrow Q$$

	A	B	C	D
0	B	B	B	D
1	A	C	D	D

My description on the debugged state machine

The un-debugged state machine only has three states. It does make sure that the string starts with input '0' instead of 1. It directly send to a dead state. Meanwhile, the debugged state machine just waits until the correct input is produced in order for it to be sent to the next state and at the same time make sure that the following states only accept input '1' as valid until it reaches the final state which will accept either 1, 0. States B and C makes sure that there are two '1' at least because final state might produce input '0' consistently.